

Atividade pratica: LLM via OpenRouter

Fundamentos de Inteligencia Artificial - ADS

Objetivo: implementar uma pequena aplicacao que envia uma mensagem para um modelo de linguagem usando a API do OpenRouter e exibe a resposta gerada pela IA.

Modelo utilizado nesta atividade: openai/gpt-oss-120b:free

O que eu espero que voces entendam

- Como uma aplicacao conversa com uma API externa.
- Como uma chave de API fica protegida em um arquivo .env.
- Como o prompt enviado ao modelo influencia diretamente a resposta.
- Como transformar um LLM em uma funcionalidade dentro de um sistema.

Importante: esta e uma atividade de prototipo. O free tier pode ter limites de uso. Evitem ficar executando o mesmo teste muitas vezes sem necessidade.

Entrega da atividade

Voce deve entregar um projeto simples, mas funcional. Nao e suficiente apenas copiar o codigo e executar. O projeto precisa ter uma proposta de uso.

Item	O que precisa existir
Objetivo	Explique para que serve o seu projeto.
Entrada	O usuario precisa informar algum texto, pergunta, codigo, tema ou tarefa.
Processamento	O sistema deve enviar essa entrada para o modelo via OpenRouter.
Saida	O sistema deve exibir uma resposta util para o usuario.
README	Explique como instalar, configurar a chave e executar.

Exemplos de projetos

- Assistente que gera questoes de revisao para uma prova.
 - Tutor que explica um conteudo de programacao em linguagem simples.
 - Analizador de mensagens que classifica urgencia, duvida, reclamacao ou elogio.
 - Revisor de codigo que aponta erros e melhorias.
 - Assistente de requisitos que transforma uma ideia em funcionalidades.
- Regra:** nao quero apenas um chat generico. Quero uma aplicacao com uma finalidade clara.

Modo simples: primeiro teste no terminal

Este modo serve para validar a chave e entender a chamada de API. Ele não abre navegador e não cria servidor. Ele apenas executa o arquivo `index.js`, chama o modelo uma vez e imprime a resposta no terminal.

Passo a passo

1. Criar uma pasta para o projeto.
2. Criar manualmente o arquivo `package.json`.
3. Executar `npm install`.
4. Criar o arquivo `index.js`.
5. Criar o arquivo `.env`.
6. Executar `npm start`.

Comandos iniciais

```
mkdir atividade-openrouter-simples  
cd atividade-openrouter-simples  
npm install
```

Resultado esperado: o terminal deve mostrar uma resposta da IA. Depois disso, o script termina.

Modo simples - arquivo package.json

Crie este arquivo manualmente na raiz da pasta do projeto.

package.json

```
{
  "type": "module",
  "scripts": {
    "start": "node index.js"
  },
  "dependencies": {
    "dotenv": "latest"
  }
}
```

Depois de criar o package.json, execute `npm install`. Esse comando instala as dependências descritas no arquivo.

Modo simples - arquivo .env

Crie o arquivo .env na raiz do projeto. Ele deve guardar a chave de API.

.env

```
OPENROUTER_API_KEY=sua_chave_aqui
```

Nunca coloque a chave dentro do index.js. A chave deve ficar no .env, que não deve ser publicado em repositórios públicos.

Formato correto

- Sem aspas.
- Sem espaço antes ou depois do sinal de igual.
- O nome da variável precisa ser exatamente OPENROUTER_API_KEY.

Modo simples - arquivo index.js

Copie o bloco inteiro para o arquivo index.js

index.js

```
import "dotenv/config";

const API_KEY = process.env.OPENROUTER_API_KEY;
const MODEL = "openai/gpt-oss-120b:free";

if (!API_KEY) {
  console.error("Erro: crie o arquivo .env com OPENROUTER_API_KEY.");
  process.exit(1);
}

async function chamarLLM() {
  const response = await fetch("https://openrouter.ai/api/v1/chat/completions", {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${API_KEY}`,
      "Content-Type": "application/json",
      "HTTP-Referer": "http://localhost:3000",
      "X-OpenRouter-Title": "Atividade FIA ADS"
    },
    body: JSON.stringify({
      model: MODEL,
      messages: [
        {
          role: "system",
          content: "Voce e um tutor didatico para alunos de ADS. Responda com clareza, exemplos simples e sem inventar informacoes."
        },
        {
          role: "user",
          content: "Explique o que e uma API para um aluno iniciante."
        }
      ]
    },
    temperature: 0.7,
    max_completion_tokens: 500
  ))
  });

  if (!response.ok) {
    const detalhe = await response.text();
    throw new Error(`Erro na API: ${response.status} - ${detalhe}`);
  }

  const data = await response.json();
  const text = data.choices?.[0]?.message?.content;

  if (!text) {
    throw new Error("A API respondeu, mas nao retornou texto.");
  }

  console.log("\nResposta da IA:\n");
  console.log(text);
}

chamarLLM().catch((error) => {
  console.error("Falha ao chamar o OpenRouter:");
  console.error(error.message);
});
```

Modo simples - execucao e leitura do resultado

Depois de criar os arquivos, execute:

Terminal
npm start

O que deve acontecer?

- O Node executa o arquivo index.js.
- O script carrega a chave do arquivo .env.
- O script envia uma pergunta fixa para o modelo.
- O terminal imprime a resposta da IA.
- Depois disso, o programa finaliza.

Erros comuns

Erro	Possivel causa
Chave nao encontrada	Arquivo .env ausente ou nome da variavel errado.
401	Chave invalida.
429	Limite de requisicoes atingido.
Resposta vazia	A API respondeu em formato inesperado ou houve falha temporaria.

Modo com Express: API local + pagina HTML

Este modo e mais proximo de uma aplicacao real. Aqui, criamos um servidor Node.js com Express. A pagina HTML conversa com a nossa API local, e a API local conversa com o OpenRouter.

Por que usar esse modo?

- A chave do OpenRouter fica protegida no back-end.
- O navegador nao acessa diretamente a chave.
- Fica mais facil transformar a ideia em um projeto com interface.

Passo a passo

1. Criar uma pasta para o projeto.
2. Criar manualmente o package.json.
3. Executar `npm install`.
4. Criar o arquivo `server.js`.
5. Criar o arquivo `.env`.
6. Criar a pasta `public`.
7. Criar o arquivo `public/index.html`.
8. Executar `npm start`.

Comandos iniciais

```
mkdir atividade-openrouter-express
cd atividade-openrouter-express
mkdir public
npm install
```


Modo com Express - arquivo package.json

Crie este arquivo manualmente na raiz da pasta do projeto.

package.json

```
{
  "type": "module",
  "scripts": {
    "start": "node server.js",
    "dev": "node server.js"
  },
  "dependencies": {
    "cors": "latest",
    "dotenv": "latest",
    "express": "latest"
  }
}
```

Depois de criar o arquivo, execute `npm install` para instalar Express, CORS e Dotenv.

Modo com Express - arquivo .env

O arquivo .env continua ficando na raiz do projeto.

.env

```
OPENROUTER_API_KEY=sua_chave_aqui
```

Atencao: o arquivo .env nao fica dentro da pasta public. Ele fica na raiz, junto do server.js e do package.json.

Modo com Express - arquivo server.js

Copie o bloco inteiro para o arquivo server.js

server.js

```
import express from "express";
import cors from "cors";
import "dotenv/config";

const app = express();
const PORT = 3000;
const API_KEY = process.env.OPENROUTER_API_KEY;
const MODEL = "openai/gpt-oss-120b:free";

if (!API_KEY) {
  console.error("Erro: configure OPENROUTER_API_KEY no arquivo .env.");
  process.exit(1);
}

app.use(cors());
app.use(express.json());
app.use(express.static("public"));

app.get("/api/status", (req, res) => {
  res.json({ status: "API local funcionando", model: MODEL });
});

app.post("/api/llm", async (req, res) => {
  try {
    const { prompt } = req.body;

    if (!prompt || prompt.trim().length === 0) {
      return res.status(400).json({ erro: "O campo prompt é obrigatório." });
    }

    if (prompt.length > 2000) {
      return res.status(400).json({ erro: "Limite: 2000 caracteres." });
    }

    const response = await fetch("https://openrouter.ai/api/v1/chat/completions", {
      method: "POST",
      headers: {
        "Authorization": `Bearer ${API_KEY}`,
        "Content-Type": "application/json",
        "HTTP-Referer": "http://localhost:3000",
        "X-OpenRouter-Title": "Projeto FIA ADS"
      },
      body: JSON.stringify({
        model: MODEL,
        messages: [
          {
            role: "system",
            content: "Você é um assistente acadêmico de ADS. Responda de forma objetiva, didática e com exemplos simples."
          },
          {
            role: "user",
            content: prompt
          }
        ],
        temperature: 0.7,
        max_completion_tokens: 700
      })
    });

    if (!response.ok) {
      const detalhe = await response.text();
      return res.status(502).json({
        erro: "Erro ao consultar o OpenRouter.",
        status: response.status,
        detalhe
      });
    }

    const data = await response.json();
    const text = data.choices?.[0]?.message?.content;

    if (!text) {
      return res.status(502).json({ erro: "Resposta vazia ou inesperada." });
    }

    res.json({ modelo: MODEL, resposta: text, uso: data.usage ?? null });
  } catch (error) {
    res.status(500).json({ erro: "Erro interno no servidor.", detalhe: error.message });
  }
});

app.listen(PORT, () => {
  console.log(`Servidor rodando em http://localhost:${PORT}`);
});
```

Modo com Express - arquivo public/index.html

Crie a pasta public e salve este arquivo dentro dela

public/index.html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8" />
  <title>Projeto LLM com OpenRouter</title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 850px; margin: 32px auto; }
    textarea { width: 100%; height: 140px; font-size: 16px; }
    button { padding: 10px 18px; font-size: 16px; cursor: pointer; }
    pre { white-space: pre-wrap; background: #f1f5f9; padding: 16px; }
  </style>
</head>
<body>
  <h1>Assistente com LLM</h1>
  <p>Digite uma pergunta ou tarefa para enviar ao modelo via API local.</p>

  <textarea id="prompt" placeholder="Ex.: Crie 3 perguntas sobre APIs REST."></textarea>
  <br><br>
  <button onclick="enviar()">Enviar</button>

  <h2>Resposta</h2>
  <pre id="resposta">A resposta aparecera aqui.</pre>

  <script>
    async function enviar() {
      const prompt = document.getElementById("prompt").value;
      const output = document.getElementById("resposta");

      output.textContent = "Consultando a IA...";

      try {
        const response = await fetch("/api/llm", {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({ prompt })
        });

        const data = await response.json();

        if (!response.ok) {
          output.textContent = data.erro || "Erro desconhecido.";
          return;
        }

        output.textContent = data.resposta;
      } catch (error) {
        output.textContent = "Erro ao conectar com a API local.";
      }
    }
  </script>
</body>
</html>
```

Modo com Express - execucao e teste

Execute o servidor:

Terminal

```
npm start
```

Depois, acesse no navegador:

Navegador

```
http://localhost:3000
```

Teste alternativo usando cURL

Terminal

```
curl -X POST http://localhost:3000/api/llm \  
-H "Content-Type: application/json" \  
-d '{"prompt": "Explique o que e uma API REST com um exemplo simples."}'
```

O que deve acontecer?

- O terminal deve mostrar: Servidor rodando em http://localhost:3000.
- A pagina HTML deve abrir no navegador.
- Ao clicar em Enviar, a pagina chama /api/llm.
- O servidor chama o OpenRouter e retorna a resposta para a pagina.

Como transformar isso em projeto

A partir do exemplo com Express, voce deve adaptar o prompt, a interface e a finalidade do sistema.

Exemplo 1: tutor de logica

- Entrada: duvida do aluno.
- Prompt de sistema: responder como professor de logica de programacao.
- Saida: explicacao, exemplo em JavaScript e exercicio simples.

Exemplo 2: analisador de requisitos

- Entrada: ideia inicial de aplicativo.
- Prompt de sistema: atuar como analista de sistemas.
- Saida: funcionalidades, usuarios, regras de negocio e riscos.

Exemplo 3: revisor de codigo

- Entrada: trecho de codigo.
- Prompt de sistema: revisar como monitor de programacao.
- Saida: erros, melhorias e explicacao didatica.

O diferencial esta no prompt. Quanto melhor voces definirem o papel da IA, o tipo de entrada e o formato de resposta, melhor sera o projeto.

Checklist antes de entregar

Verificacao	OK?
O projeto executa com npm start.	
O arquivo .env foi criado corretamente.	
A chave da API nao aparece no front-end.	
A aplicacao usa o modelo openai/gpt-oss-120b:free.	
A entrada do usuario e validada.	
A resposta da IA aparece na tela ou no terminal.	
O README explica como instalar e executar.	
O projeto tem uma finalidade clara, e nao apenas um chat generico.	

Estrutura esperada no modo Express

Arquivos

atividade-openrouter-express/
package.json
server.js
.env
public/
index.html